

LC 1833 (Amazon)

Authored by: Arpit Raj, LNMIIT Jaipur | Date: October 4, 2026

Related Problems: 1833, 1122, 1636, 347

Counting Sort Concept

Example Array:

```
arr = [4, 2, 2, 8, 3, 3, 1]
```

Then in counting sort, we first count how many times each value occurs.

val (ie)	0	1	2	3	4	5	6	7	8
count	0	1	2	2	1	0	0	0	1

Then → walk through count and rebuild sorted array

So:

```
[1, 2, 2, 3, 3, 4, 8]
```

Complexity Analysis

finding max val	$O(n)$
counting occurrence of each	$O(n)$
iterating count array	$O(k)$
building final	$O(n)$
So $O(n + n + k + n)$	$O(n + k)$

Trade-offs & Constraints

$O(n \log n)$ is considered better, because counting sort is not good for large ranges (1, 2, 3, 999)
It will loop through empty lots of [4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99]

Counting Sort Templates

When values in small range:- $O(n)$

```
vector<int> cnt(max_val + 1, 0); for(int x : arr) { cnt[x]++; } for(int i = 0; i <= MAX; i++) { while(cnt[i] > 0) { // use i; cnt[i]--; } }
```

Huge range :- map $O(n \log n)$

```
map<int, int> mp; for(int x : arr) { mp[x]++; } for(auto it : mp) { int x = it.first; int freq = it.second; while(freq--) { // use x } }
```

Solving 1833

Method 1: Map Approach (Huge Range)

```
for(auto it:mp){ int price=it.first; int freq=it.second; if(coins<price){ break; } int canBuy=min(freq,coins/price); ans+=canBuy; coins-=canBuy*price; }
```

Solving 1833 (Continued)

Method 2: Optimal Counting Sort Approach

```
class Solution { public: int maxIceCream(vector<int>& costs, int coins) { int
n=costs.size(); int maxu=*max_element(costs.begin(),costs.end()); int ans=0;
vector<int> cnt(maxu+1,0); for(int cost:costs){ cnt[cost]++; } for(int
price=0;price<=maxu;price++){ if(cnt[price]==0) continue; if(coins<price)
break; int canbuy=min(cnt[price],coins/price); coins-=canbuy*price;
ans+=canbuy; } return ans; } };
```